

ARQUITETURAS DE COMPUTADORES

TURMA A1

TÓPICO 2 – MICROARQUITETURA (AULA 15)

Prof. Vinod Rebello, vinod@ic.uff.br, Sala 413

Tópico 2: A grande pergunta



Se $(i \leq n)$ então $m := a[i];$

- Você tem o papel de ser um compilador
- (Opção A) Traduz diretamente para uma sequência de *macroinstruções* correspondentes
 - Qual é o desempenho e custo do macroprograma?

Nossa ISA

ISA MAC-1 (nível 2)

- 23 Macroinstruções
- de 16 bits (coluna 1)
- Opcode crescente (4 ou 7 bits), não de tamanho fixo
- Três tipos de endereçamento:
 - Endereçamento direto
 - Endereçamento imediato
 - Endereçamento indexado
- Um operando de 12 bits ou 8 bits ou sem operando
 - Não determina diretamente o número de dados necessário para a operação

	Binary	Mnemonic	Instruction	Meaning
0000	xxxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001	xxxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010	xxxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011	xxxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100	xxxxxxxxxxx	JPOS	Jump positive	If $ac \geq 0$ then $pc := x$
0101	xxxxxxxxxxx	JZER	Jump zero	If $ac = 0$ then $pc := x$
0110	xxxxxxxxxxx	JUMP	Jump	$pc := x$
0111	xxxxxxxxxxx	LOCO	Load constant	$ac := x$ ($0 \leq x \leq 4095$)
1000	xxxxxxxxxxx	LODL	Load local	$ac := m[sp + x]$
1001	xxxxxxxxxxx	STOL	Store local	$m[x + sp] := ac$
1010	xxxxxxxxxxx	ADDL	Add local	$ac := ac + m[sp + x]$
1011	xxxxxxxxxxx	SUBL	Subtract local	$ac := ac - m[sp + x]$
1100	xxxxxxxxxxx	JNEG	Jump negative	If $ac < 0$ then $pc := x$
1101	xxxxxxxxxxx	JNZE	Jump nonzero	If $ac \neq 0$ then $pc := x$
1110	xxxxxxxxxxx	CALL	Call procedure	$sp := sp - 1; m[sp] := pc; pc := x$
1111000	00000000	PSHI	Push indirect	$sp := sp - 1; m[sp] := m[ac]$
1111001	00000000	POPI	Pop indirect	$m[ac] := m[sp]; sp := sp + 1$
1111010	00000000	PUSH	Push onto stack	$sp := sp - 1; m[sp] := ac$
1111011	00000000	POP	Pop from stack	$ac := m[sp]; sp := sp + 1$
1111100	00000000	RETN	Return	$pc := m[sp]; sp := sp + 1$
1111101	00000000	SWAP	Swap ac, sp	$tmp := ac; ac := sp; sp := tmp$
1111110	yyyyyyyyy	INSP	Increment sp	$sp := sp + y$ ($0 \leq y \leq 255$)
1111111	yyyyyyyyy	DESP	Decrement sp	$sp := sp - y$ ($0 \leq y \leq 255$)

xxxxxxxxxxx is a 12-bit machine address; in column 4 it is called x .
 yyyyyyy is an 8-bit constant; in column 4 it is called y .

Se $(i \leq n)$ então $m := a[i]$;

- Traduz diretamente para uma sequência de **macroinstruções** correspondentes:

- Duas de várias implementações:

- Analisando a 1ª

- Custo: 8×16 bits

- Desempenho:

- $(3 \text{ ou } 8) \times \sim 9$ ciclos

- 26 ou $(27 + 47)$ ciclos

LODD &n
SUBD &i
JNEG saída

LODD &i
SUBD &n
JZER dentro
JPOS saída

LOCO &a
ADDD &i
PSHI
POP
STOD &m

saída: ...

dentro: LOCO &a
ADDD &i
PSHI
POP
STOD &m

saída: ...

Se $(i \leq n)$ então $m := a[i];$

- Você tem o papel de ser um compilador de novo
- (Opção B) Traduz diretamente para um conjunto de *microinstruções* correspondentes
 - Qual é o desempenho e custo do microprograma?

Se $(i \leq n)$ então $m := a[i]$;

- Você tem o papel de ser um compilador de novo
- (Opção B) Traduz diretamente para um conjunto de *microinstruções* correspondentes
 - Qual é o desempenho e custo do microprograma?
- Este é um exercício puramente hipotético
 - A arquitetura MIC-1, como está, não suporta este estilo de execução
 - Este exercício é só para validar a hipótese de Von Neumann

O Microprograma de MIC-1

- Este microprograma tem 79 microinstruções de 32 bits

0: mar:=pc; rd;	Busca	40: tir:=lshift(tir); if n then goto 46;	
1: pc:=pc + 1; rd;	Busca	41: alu:=tir; if n then goto 44;	
2: jr:=mbr; if n then goto 28;	Busca/Identifica	42: alu:=ac; if n then goto 22;	{1100 = JNEG}
3: tir:=lshift(ir + jr); if n then goto 19;	Identifica	43: goto 0;	
4: tir:=lshift(tir); if n then goto 11;	Identifica	44: alu:=ac; if z then goto 0;	{1101 = JNZE}
5: alu:=tir; if n then goto 9;	Identifica	45: pc:=band(ir.amask); goto 0;	
6: mar:=ir; rd;	Executa	46: tir:=lshift(tir); if n then goto 50;	
	{0000 = LODD}	47: sp:=sp + (-1);	{1110 = CALL}
7: rd;	Executa	48: mar:=sp; mbr:=pc; wr;	
8: ac:=mbr; goto 0;	Executa	49: pc:=band(ir.amask); wr; goto 0;	
9: mar:=ir; mbr:=ac; wr;	{0001 = STOD}	50: tir:=lshift(tir); if n then goto 65;	
10: wr; goto 0;	Executa	51: tir:=lshift(tir); if n then goto 59;	
11: alu:=tir; if n then goto 15;	Identifica	52: alu:=tir; if n then goto 56;	
12: mar:=ir; rd;	{0010 = ADDD}	53: mar:=ac; rd;	{1111-0000 = PSHI}
13: rd;	Executa	54: sp:=sp + (-1); rd;	
14: ac:=mbr + ac; goto 0;	Executa	55: mar:=sp; wr; goto 10;	
15: mar:=ir; rd;	{0011 = SUBD}	56: mar:=sp; sp:=sp + 1; rd;	{1111-0010 = POPI}
16: ac:=ac + 1; rd;	Executa	57: rd;	
17: a:=inv(mbr);	Executa	58: mar:=ac; wr; goto 10;	
18: ac:=ac + a; goto 0;	Executa	59: alu:=tir; if n then goto 62;	
19: tir:=lshift(tir); if n then goto 25;	Identifica	60: sp:=sp + (-1);	{1111-0100 = PUSH}
20: alu:=tir; if n then goto 23;		61: mar:=sp; mbr:=ac; wr; goto 10;	
21: alu:=ac; if n then goto 0;	{0100 = JPOS}	62: mar:=sp; sp:=sp + 1; rd;	{1111-0110 = POP}
22: pc:=band(ir.amask); goto 0;		63: rd;	
23: alu:=ac; if z then goto 22;	{0101 = JZER}	64: ac:=mbr; goto 0;	
24: goto 0;		65: tir:=lshift(tir); if n then goto 73;	
25: alu:=tir; if n then goto 27;		66: alu:=tir; if n then goto 70;	
26: pc:=band(ir.amask); goto 0;	{0110 = JUMP}	67: mar:=sp; sp:=sp + 1; rd;	{1111-1000 = RETN}
27: ac:=band(ir.amask); goto 0;	{0111 = LOCO}	68: rd;	
28: tir:=lshift(ir + jr); if n then goto 40;		69: pc:=mbr; goto 0;	
29: tir:=lshift(tir); if n then goto 35;		70: a:=ac;	{1111-1010 = SWAP}
30: alu:=tir; if n then goto 33;		71: ac:=sp;	
31: a:=ir + sp;	{1000 = LODL}	72: sp:=a; goto 0;	
32: mar:=a; rd; goto 7;		73: alu:=tir; if n then goto 76;	
33: a:=ir + sp;	{1001 = STOL}	74: a:=band(ir.smask);	{1111-1100 = INSP}
34: mar:=a; mbr:=ac; wr; goto 10;		75: sp:=sp + a; goto 0;	
35: alu:=tir; if n then goto 38;		76: a:=band(ir.smask);	{1111-1110 = DESP}
36: a:=ir + sp;	{1010 = ADDL}	77: a:=inv(a);	
37: mar:=a; rd; goto 13;		78: a:=a + 1; goto 75;	
38: a:=ir + sp;	{1011 = SUBL}		
39: mar:=a; rd; goto 16 ;			

Se $(i \leq n)$ então $m := a[i]$;

- Traduz diretamente para uma sequência de microinstruções correspondentes:

- **Desempenho:**

- 7 ou 14 ciclos **se usa a MC**

- 5 vezes melhor



- **Custo:** 14 × 32 bits

- ~4 vezes mais p/ 1 linha



- Para um programa(s) inteiro?

- μprograma só custa 316 bytes

```
mar := &n; rd      # busca n
rd;                # 2cicl p/ler
ac := mbr;         # ac recebe n
mar := &i; rd      # busca i
rd; ac := ac + 1   # subtração 1
a := inv(mbr);     # subtração 2
alu := ac + a; if n goto saída;
rd;                # MAR já tem &i
ac := &a; rd;      # carreg. end. a
ac := mbr + ac;    # end. a[i]
mar := ac; rd;     # busca a[i]
rd;
ac := mbr;
mar := &m; mbr := ac; wr;
wr;                # atualizar m
saída: ...
```

Se $(i \leq n)$ então $m := a[i]$;

- Traduz diretamente para uma sequência de microinstruções correspondentes:

- **Desempenho:**

- 7 ou 14 ciclos **se usa a MC**

- 5 vezes melhor



- **Custo:** 14 × 32 bits

- ~4 vezes mais p/ 1 linha
- Para um programa(s) inteiro?
 - μprograma só custa 316 bytes



```
mar := &n; rd      # busca n
rd;                # 2cicl p/ler
ac := mbr;         # ac recebe n
mar := &i; rd      # busca i
rd; ac := ac + 1   # subtração 1
a := inv(mbr);     # subtração 2
alu := ac + a; if n goto saída;
rd;                # MAR já tem &i
ac := &a; rd;      # carreg. end. a
ac := mbr + ac;    # end. a[i]
mar := ac; rd;     # busca a[i]
rd;
ac := mbr;
mar := &m; mbr := ac; wr;
wr;                # atualizar m
saída: ...
```

Tópico 2: A grande pergunta



Tópico 2: A grande pergunta

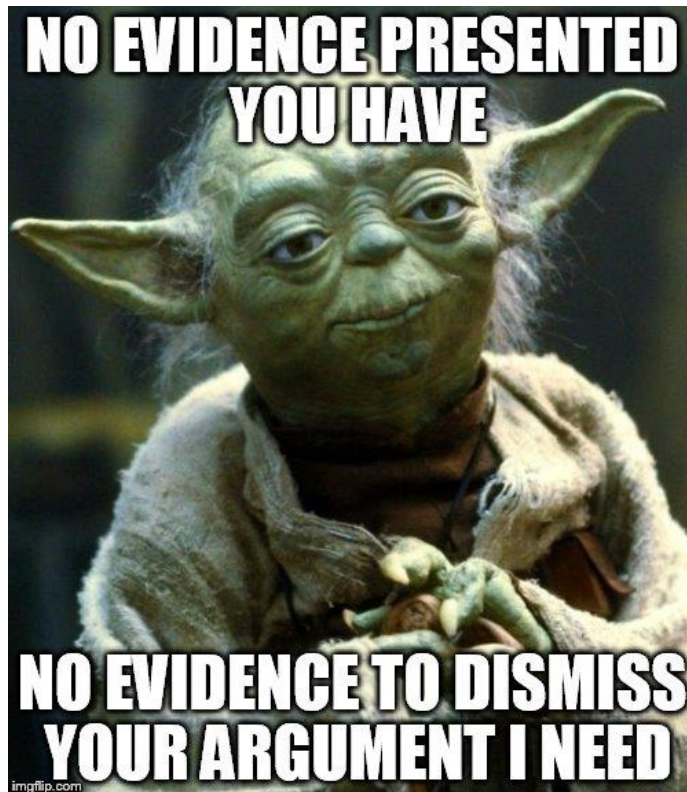
- Nossa questão tornou-se uma de
 - Macro-programação versus Micro-programação ou, em outras palavras, ...
 - Será que o compilador deve gerar uma saída para nível 1 ao invés de nível 2?
 - *"Ser ou não ser? Eis a questão."* O início do solilóquio de Príncipe Hamlet da peça *Hamlet* de William Shakespeare
 - Também é título de um conto escrito por Machado de Assis, em 1876.
 - Então, fazer ou não fazer?

Macro- x Micro-programação

- Um compilador deve gerar uma saída para nível 1 ao invés de nível 2?
 - Na prática, seria inviável devido às complexidades das linguagens e arquiteturas e por ser uma solução ineficiente, cara, lenta e de difícil implementação!

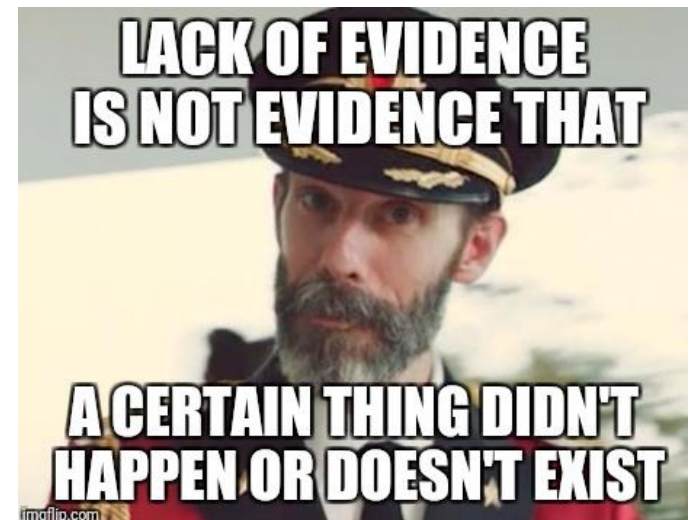
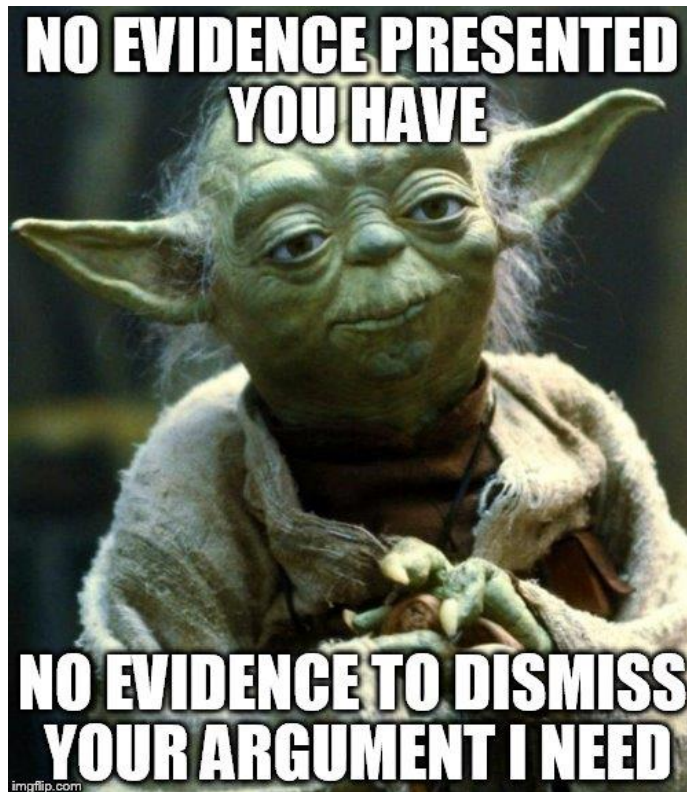
Macro- x Micro-programação

- Um compilador deve gerar uma saída para nível 1 ao invés de nível 2?
 - Na prática, seria inviável devido às complexidades das linguagens e arquiteturas e por ser uma solução ineficiente, cara, lento e de difícil implementação!



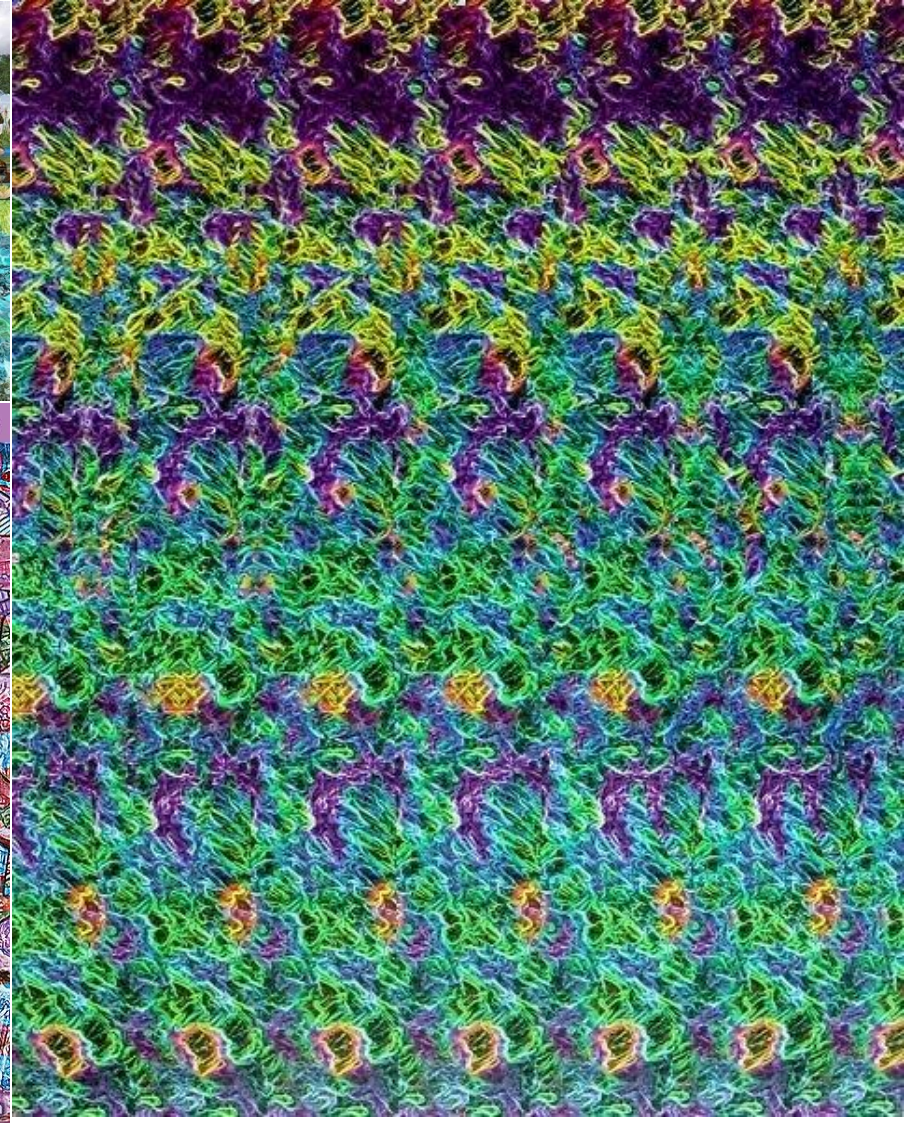
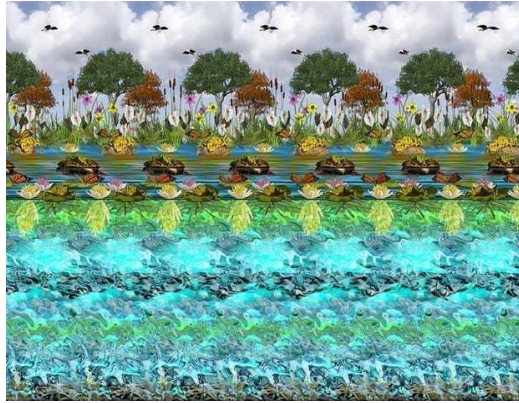
Macro- x Micro-programação

- Um compilador deve gerar uma saída para nível 1 ao invés de nível 2?
 - Na prática, seria inviável devido às complexidades das linguagens e arquiteturas e por ser uma solução ineficiente, cara, lento e de difícil implementação!



Está procurando no lugar certo?

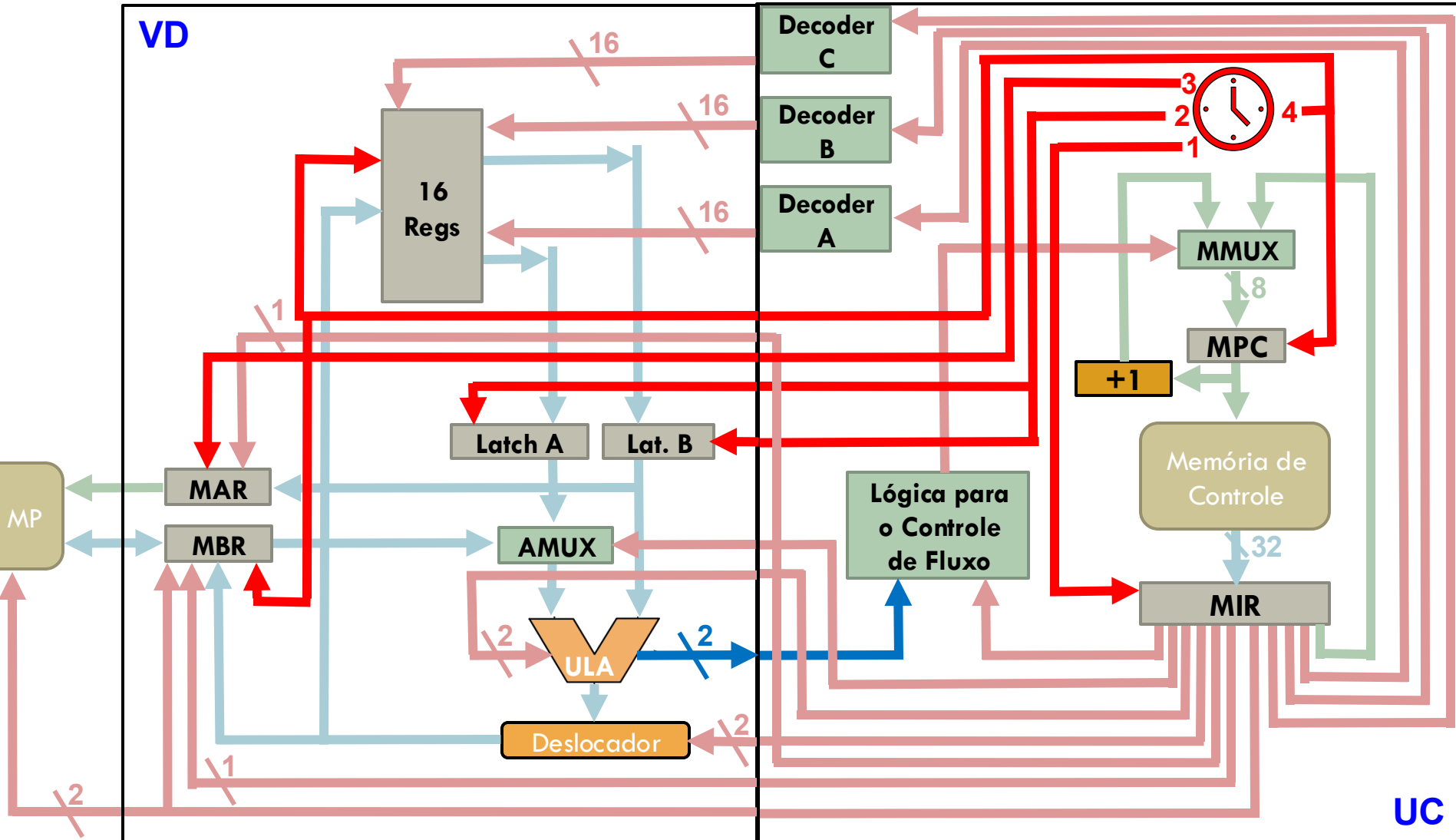
**Lembra
Aula 8?**



Can you find what's hidden in this stereogram using your 3D vision? Viewing instructions: hide3d.com/how



Quem carrega a(s) memória(s)?



Tópico 2: A grande pergunta



Se $(i \leq n)$ então $m := a[i]$;

- Traduz diretamente para uma sequência de microinstruções correspondentes:

- **Desempenho:**

- 7 ou 14 ciclos **se usa a MC**

- 5 vezes melhor



- **Se usa a MP**, tão lento quanto

- **Custo:** 14 × 32 bits

- ~4 vezes mais p/ 1 linha



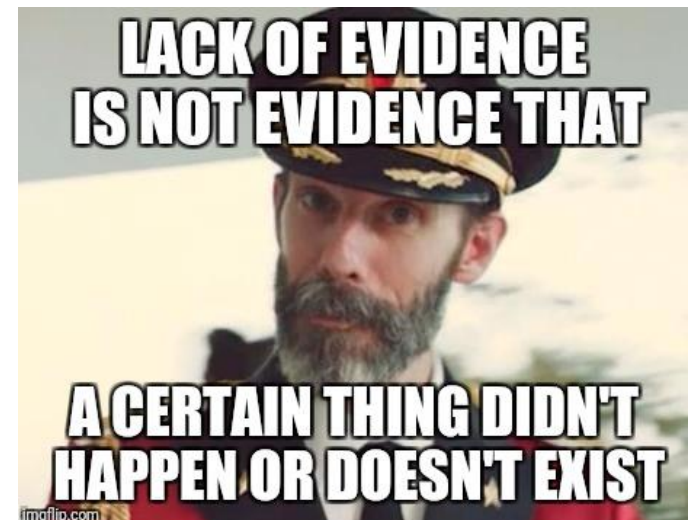
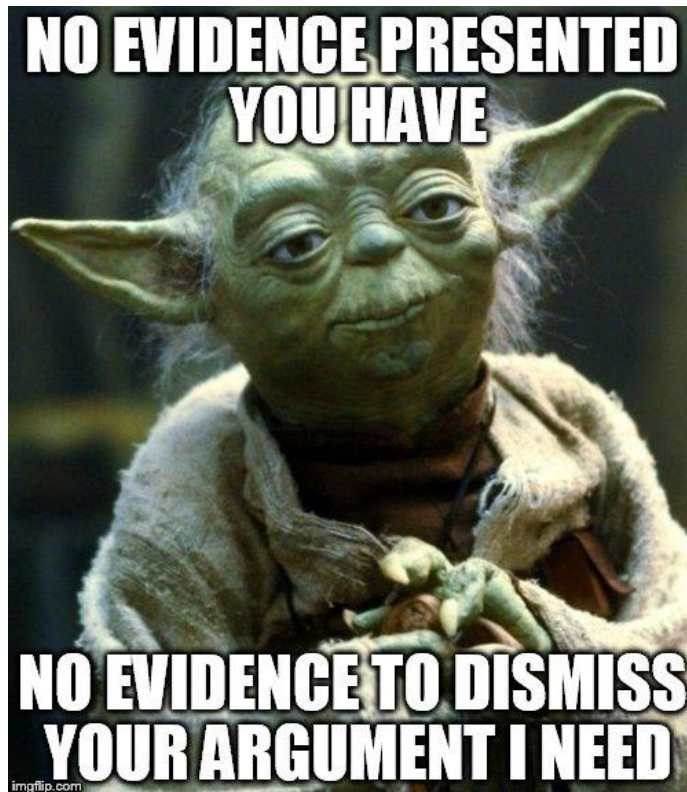
- Para um programa(s) inteiro?

- μprograma só custa 316 bytes

```
mar := &n; rd      # busca n
rd;                # 2cicl p/ler
ac := mbr;         # ac recebe n
mar := &i; rd      # busca i
rd; ac := ac + 1   # subtração 1
a := inv(mbr);     # subtração 2
alu := ac + a; if n goto saída;
rd;                # MAR já tem &i
ac := &a; rd;      # carreg. end. a
ac := mbr + ac;    # end. a[i]
mar := ac; rd;     # busca a[i]
rd;
ac := mbr;
mar := &m; mbr := ac; wr;
wr;                # atualizar m
saída: ...
```

Macro- x Micro-programação

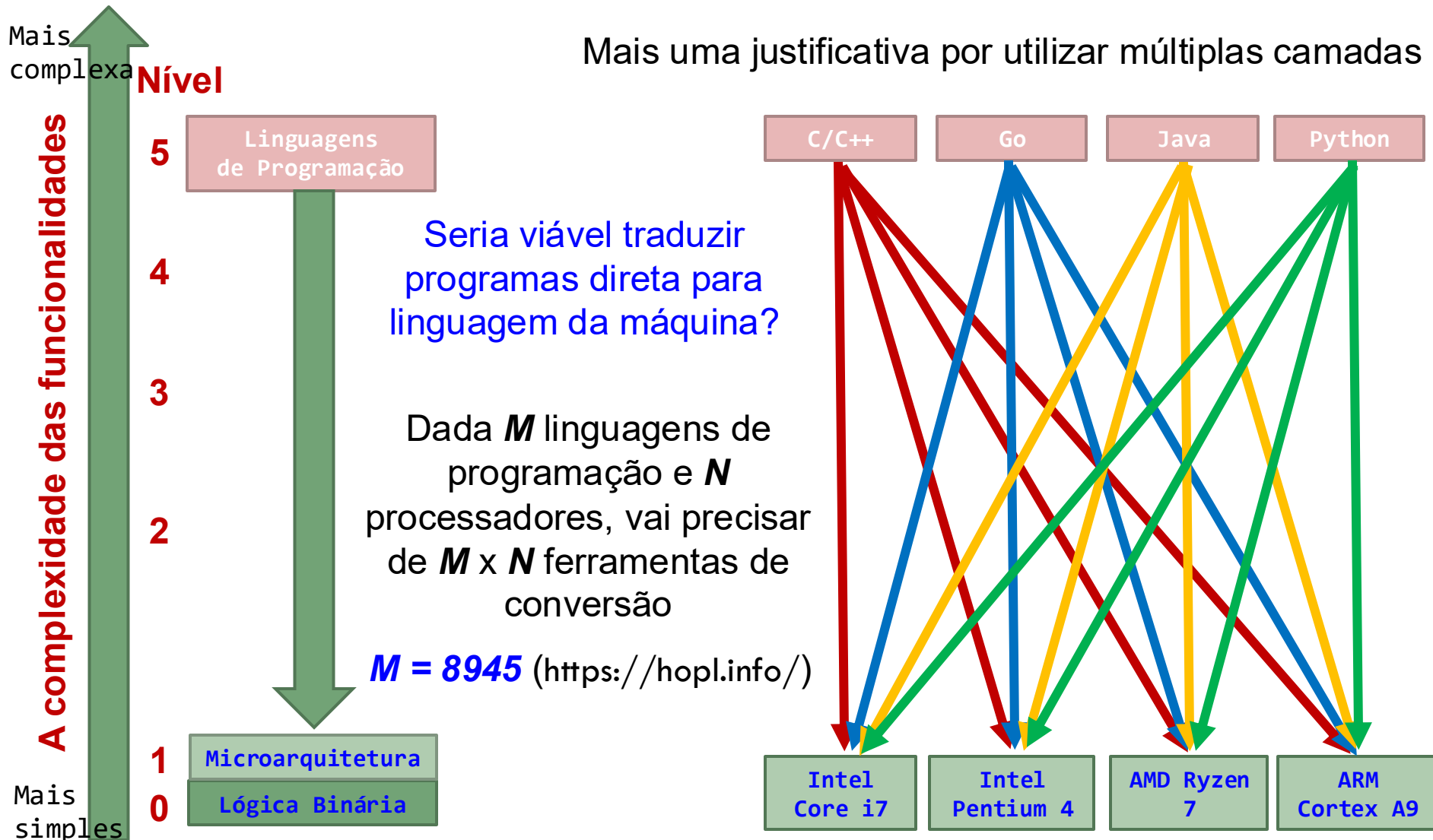
- Um compilador deve gerar uma saída para nível 1 ao invés de nível 2?
 - Na prática, seria inviável devido às complexidades das linguagens e arquiteturas e por ser uma solução ineficiente, cara, lento e de difícil implementação!



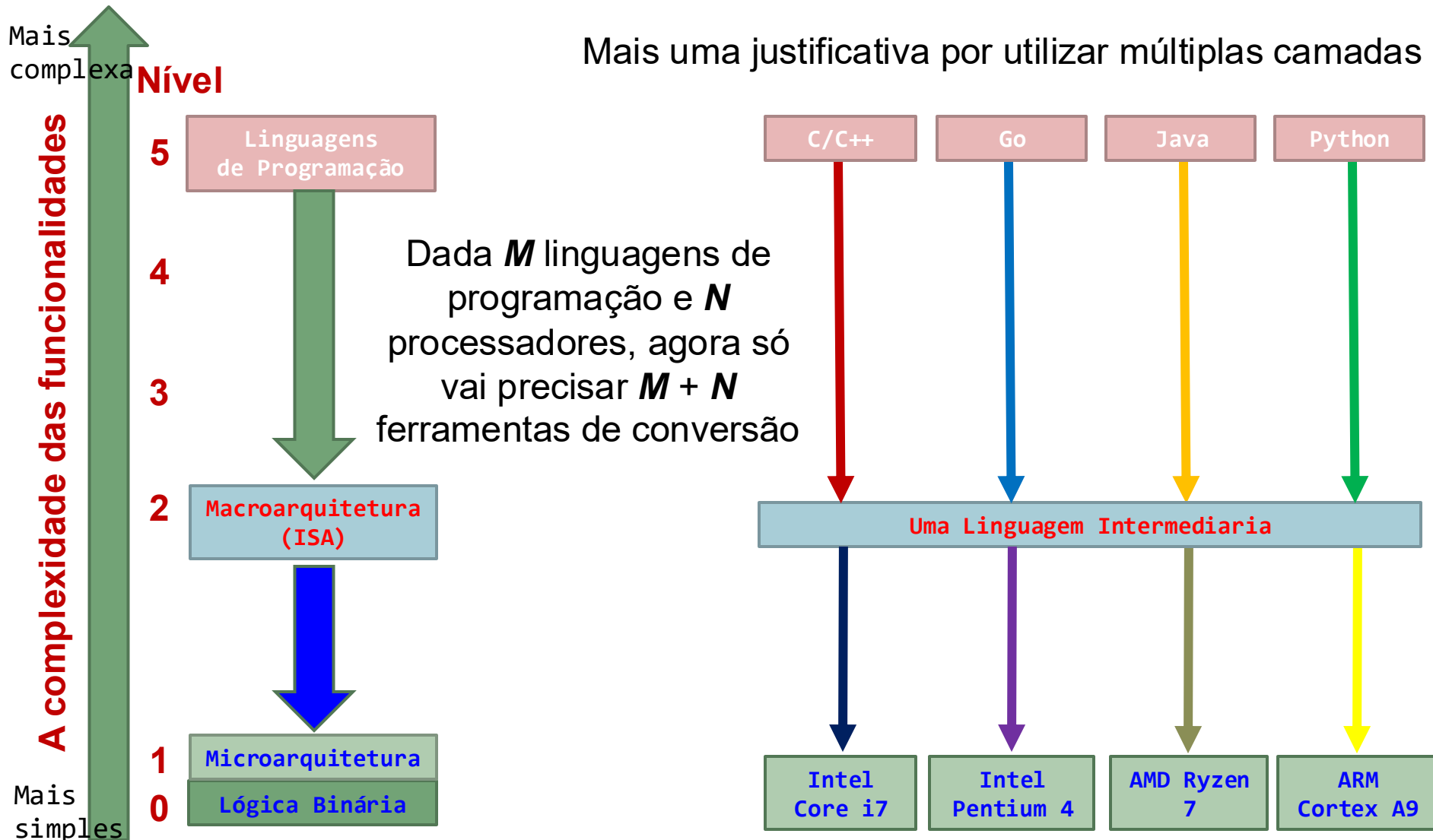
Macro- x Micro-programação

- Um compilador deve gerar uma saída para nível 1 ao invés de nível 2?
 - Na prática, seria inviável devido às complexidades das linguagens e arquiteturas e por ser uma solução ineficiente, cara, lenta e de difícil implementação!
 - Enquanto sem a interpretação, a execução direto de μ insts poderia **aparecer de ser mais rápido**. Na verdade ...
 - Vai precisar de mais memória para guardar o programa: um MC grande é muito caro; além disso, deve ser **acessível** e possível **escrever** nela; [**Custo** ✗]
 - Se usa MP, por ser mais lento, a execução fica mais lento [**Desempenho** ✗]
 - **Compilador** precisa explorar a μ arq., μ insts, os registradores, a concorrência, etc.
 - Número de μ insts é bem maior do que macroinsts, mais complexidade [**Facilidade** ✗]
 - Cada nova arquitetura tem novas μ insts $\rightarrow$ um novo compilador, e mais custos
 - Lembre-se da quantidade compiladores que serão necessários: **$m \times n$** invés de **$m + n$**

A arquitetura em múltiplas dimensões



A arquitetura em múltiplas dimensões



Tópico 2: A grande pergunta



O custo de interpretação

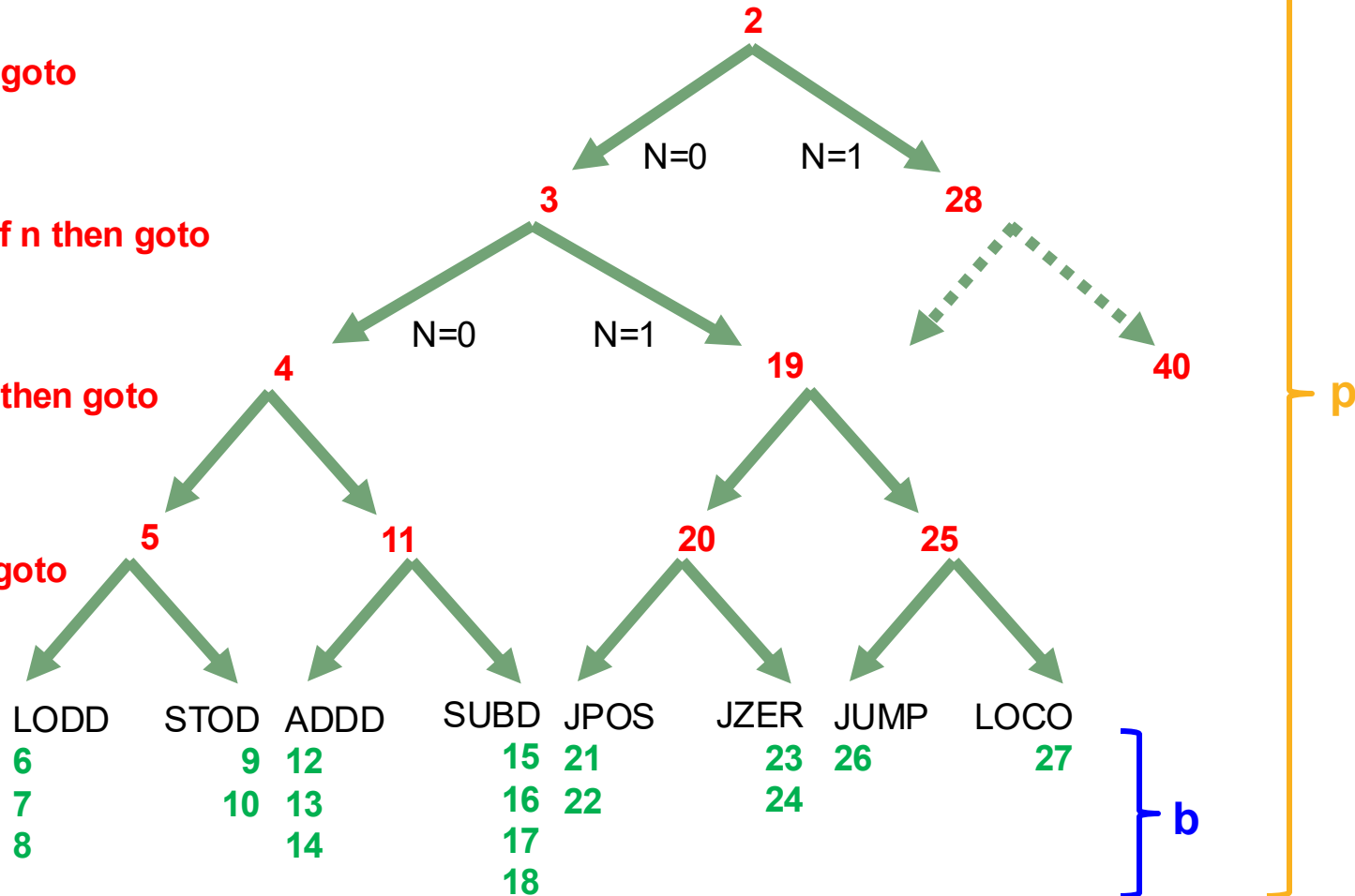
- Pode ser zero? Será que **p** pode aproximar **b**? $\frac{0}{1}$ μinsts de busca

ir := mbr; if n then goto

tir := lshift(ir + ir); if n then goto

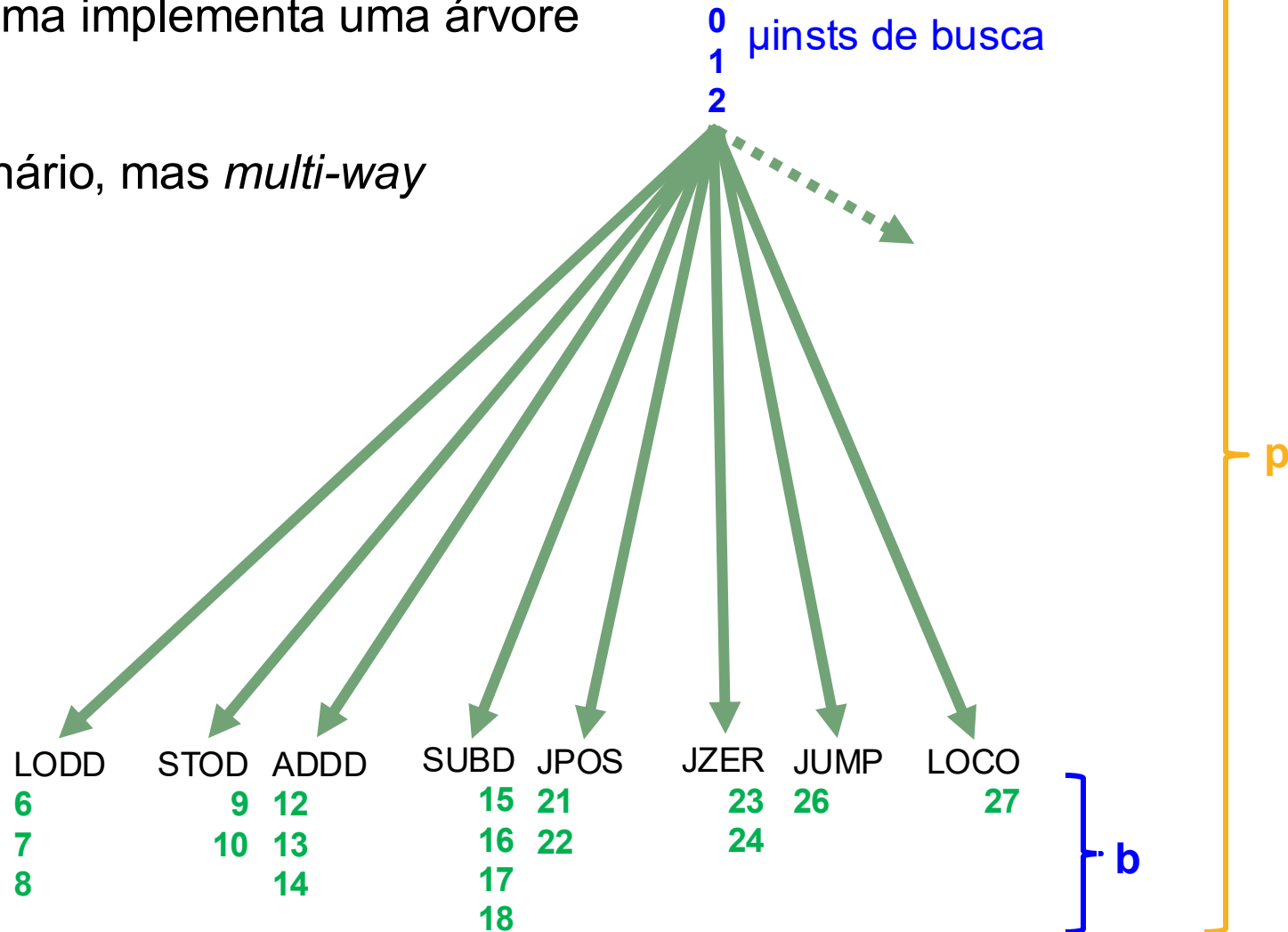
tir := lshift(tir); if n then goto

alu := tir; if n then goto

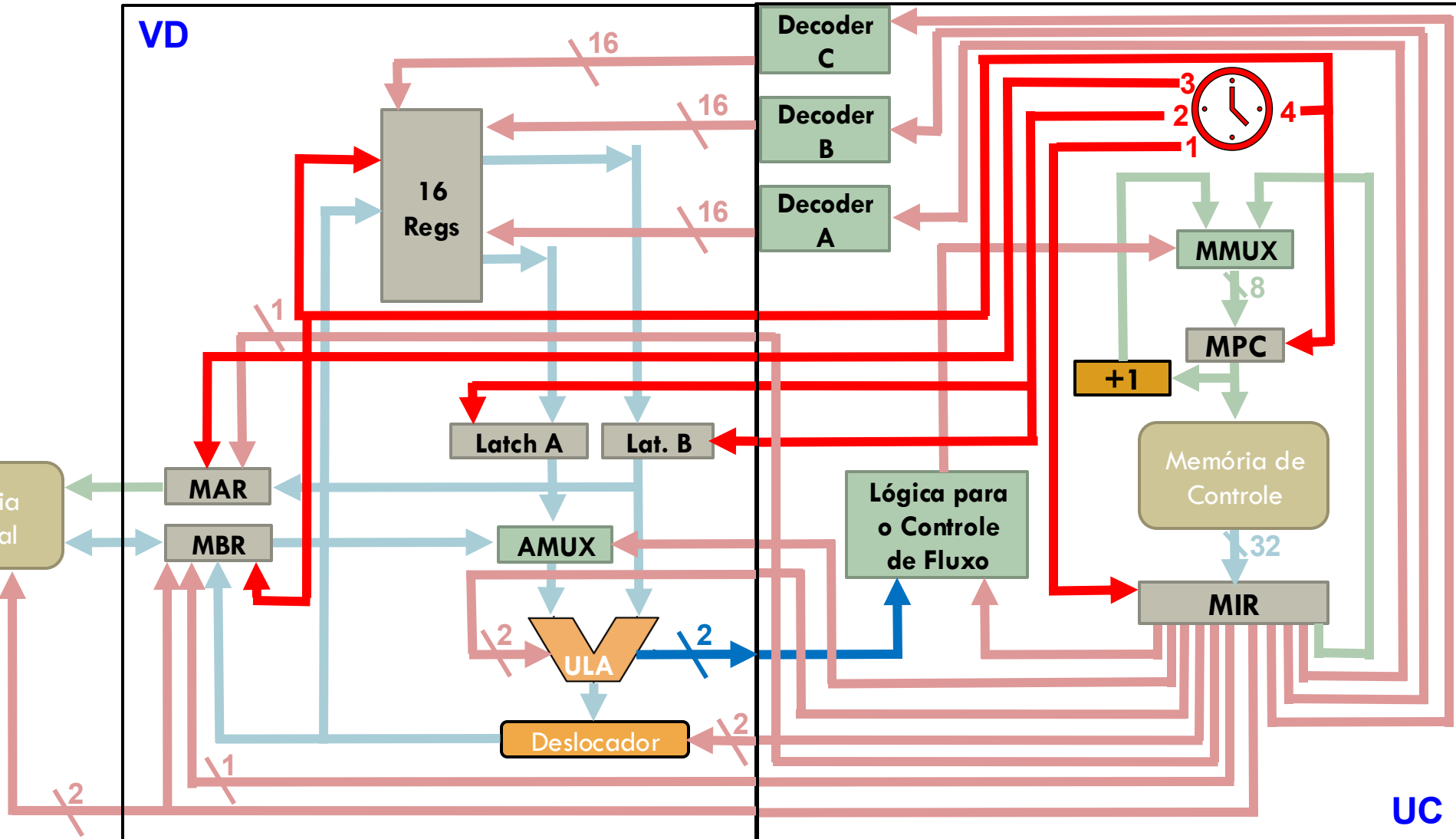


Identificação usando um "Case"?

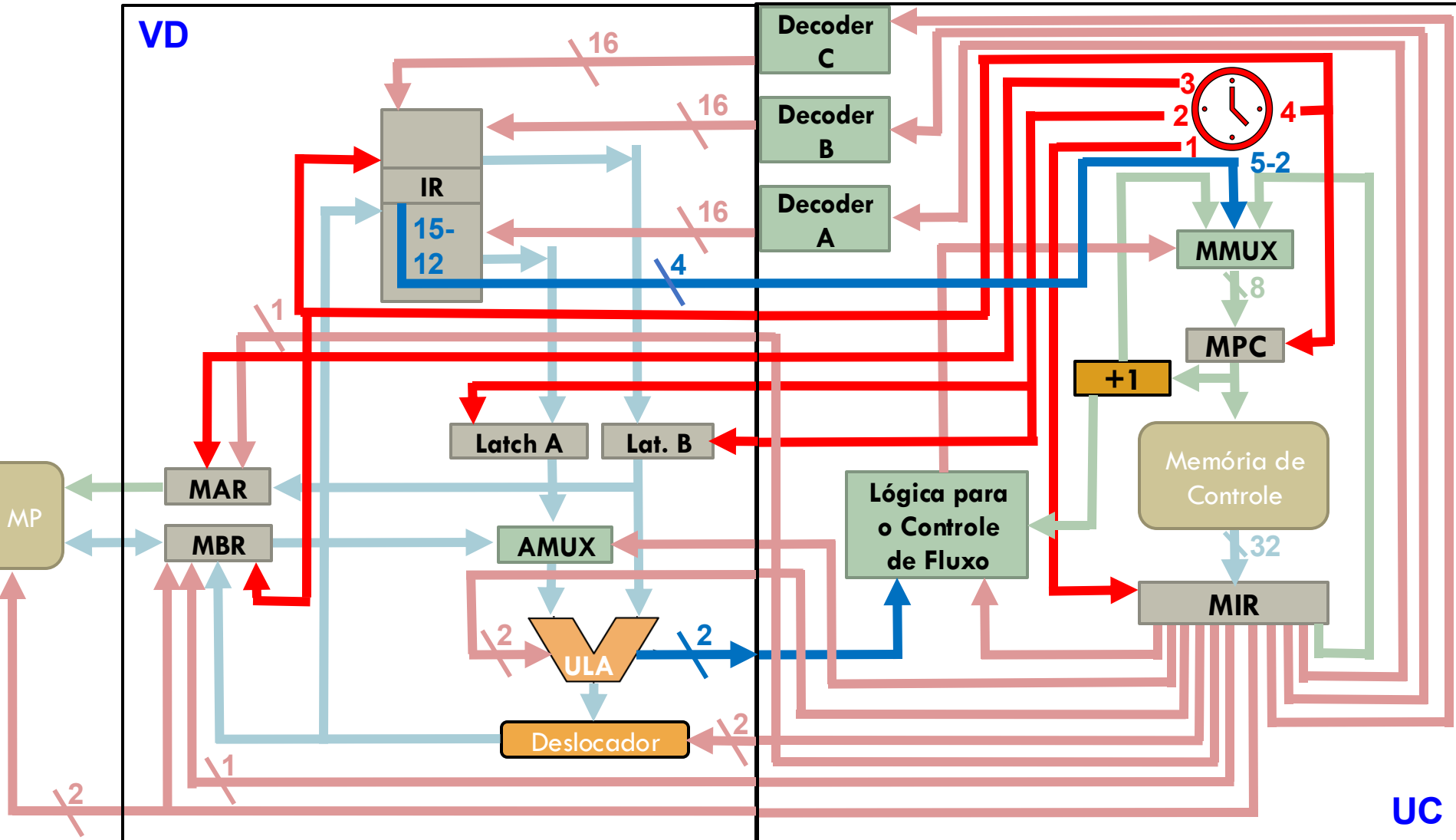
- O microprograma implementa uma árvore de decisão:
- Não é mais binário, mas *multi-way*



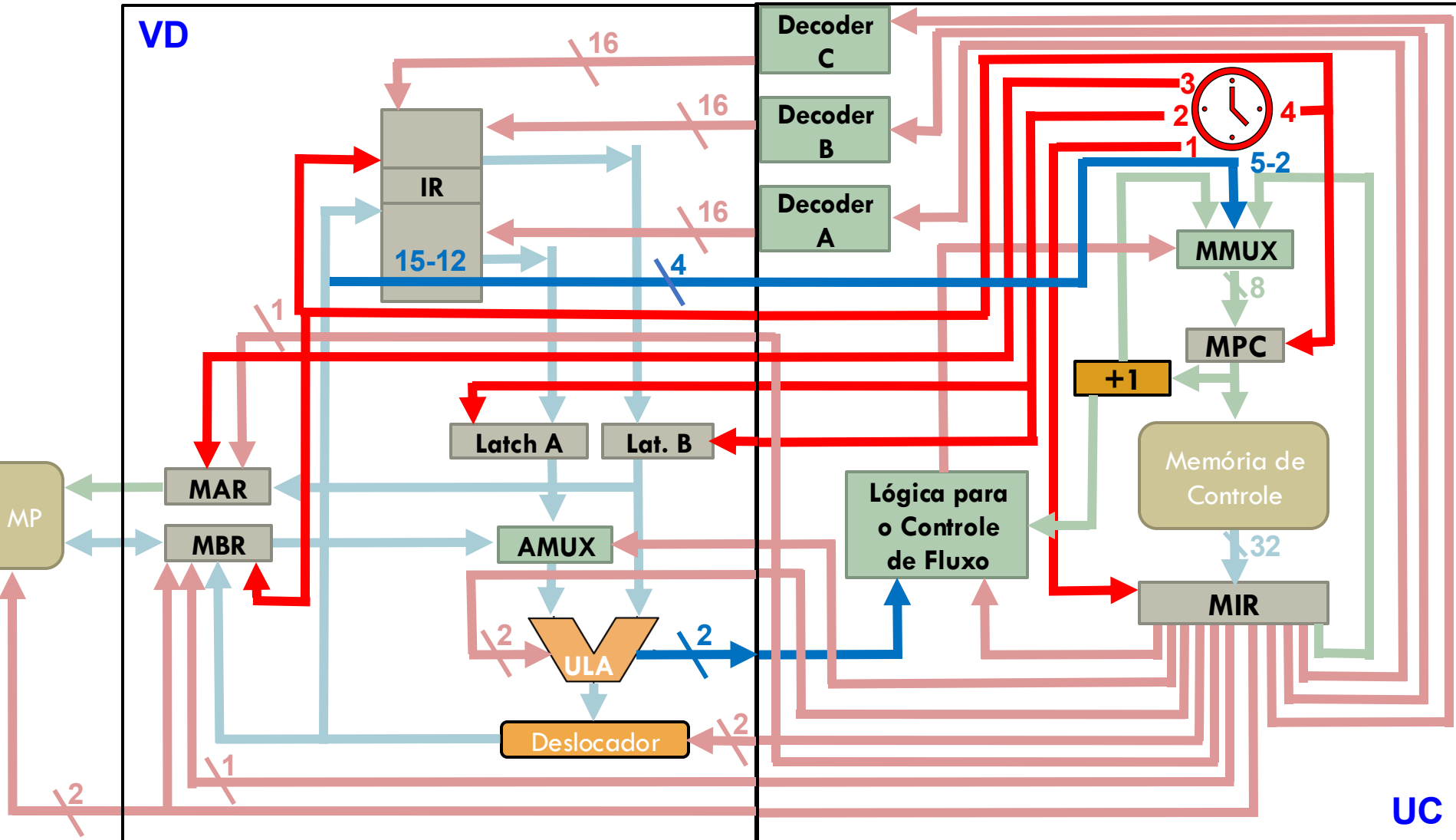
O projeto do processador MIC-1



Implementação do "Case" no MIC-1



Implementação do "Case" no MIC-1



Identificando a macroinstrução

□ Microprograma original:

- 0: `mar := pc; rd;`
- 1: `pc := pc + 1; rd;`
- 2: `ir := mbr; if n then goto 28;`
- 3: `tir := lshift(ir + ir); if n then goto 19;`
- 4: `tir := lshift(tir); if n then goto 11;`
- 5: `alu := tir; if n then goto 9;`
- 6: `mar := ir; rd;`
- 7: `rd;`
- 8: `ac := mbr; goto 0;`
- 9: `mar := ir; mbr := ac; wr;`
- 10: `wr; goto 0;`
- 11: ...

□ Microprograma com suporte h'ware:

- 0: `mar := ir; rd;` [`'0000'` = LODD]
- 1: `rd;`
- 2: `ac := mbr; goto 252;`
- 3:
- 4: `mar := ir; mbr := ac; wr;`
- 5: `wr; goto 252;` [`'0001'` = STOD]
- 6:
- 60: `tir := lshift(ir+ir);`
- 61: `tir := lshift(tir+tir);`
- 62: `tir := lshift(tir); if n then goto 77;`
- 63: ... 90: `μinsts originais 51 a 78`
- 252: `mar := pc; rd;`
- 253: `pc := pc + 1; rd;`
- 254: `ir := mbr;` [mpc = 255]